



Stock Span Problem

Last Updated : 29 Jul, 2026

The stock span problem is a financial problem where we have a series of daily price quotes for a stock and we need to calculate the span of stock price for all days.

Given an array `arr[]` representing daily stock prices, the stock span for the **i-th** day is the number of consecutive days up to day *i* (including day *i* itself) for which the price of the stock is **less than or equal** to the price on day *i*. Return the span of stock prices for each day in the given sequence.

Examples:

Input: `arr[] = [100, 80, 90, 120]`

Output: `[1, 1, 2, 4]`

Explanation: Consider each day one by one:

Day 1 (100): Span = 1.

Day 2 (80): Span = 1 because the previous price (100) is greater than 80.

Day 3 (90): Span = 2 because $80 \leq 90$, but $100 > 90$.

Day 4 (120): Span = 4 because 100, 80, and 90 are all less than or equal to 120.

Hence, the output is `[1, 1, 2, 4]`.

Input: `arr[] = [10, 4, 5, 90, 120, 80]`

Output: `[1, 1, 2, 4, 5, 1]`

Explanation: Consider each day one by one:

Day 1 (10): Span = 1.

Day 2 (4): Span = 1 because the previous price (10) is greater than 4.

Day 3 (5): Span = 2 because $4 \leq 5$, but $10 > 5$.

Day 4 (90): Span = 4 because 10, 4, and 5 are all less than or equal to 90.

Day 5 (120): Span = 5 because all previous prices are less than or equal to 120.

Day 6 (80): Span = 1 because the previous price (120) is greater than 80.

Hence, the output is `[1, 1, 2, 4, 5, 1]`.

Table of Content

- [\[Naive Approach\] Using Nested Loop - \$O\(n^2\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Expected Approach\] Using Stack - \$O\(n\)\$ Time and \$O\(n\)\$ Space](#)

[Naive Approach] Using Nested Loop - $O(n^2)$ Time and $O(1)$ Space

The idea is to check, for each day, how many consecutive previous days had stock prices less than or equal to the current day's price. This can be done by moving leftwards from the current index until a higher price is found or the beginning of the array is reached.

C++

Java

Python

C#

JavaScript

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> calculateSpan(vector<int>& arr) {

    int n = arr.size();
    vector<int> span(n, 1);

    // Calculate span for each day
    for (int i = 1; i < n; i++) {

        // Traverse left while the next element
        // on the left is smaller than arr[i]
        for (int j = i - 1; (j >= 0)
            && (arr[i] >= arr[j])); j--) {
            span[i]++;
        }
    }

    return span;
}

int main() {

    vector<int> arr = {10, 4, 5, 90, 120, 80};
    vector<int> span = calculateSpan(arr);
    for (int x : span) {
        cout << x << " ";
    }
    return 0;
}
```

Output

1 1 2 4 5 1

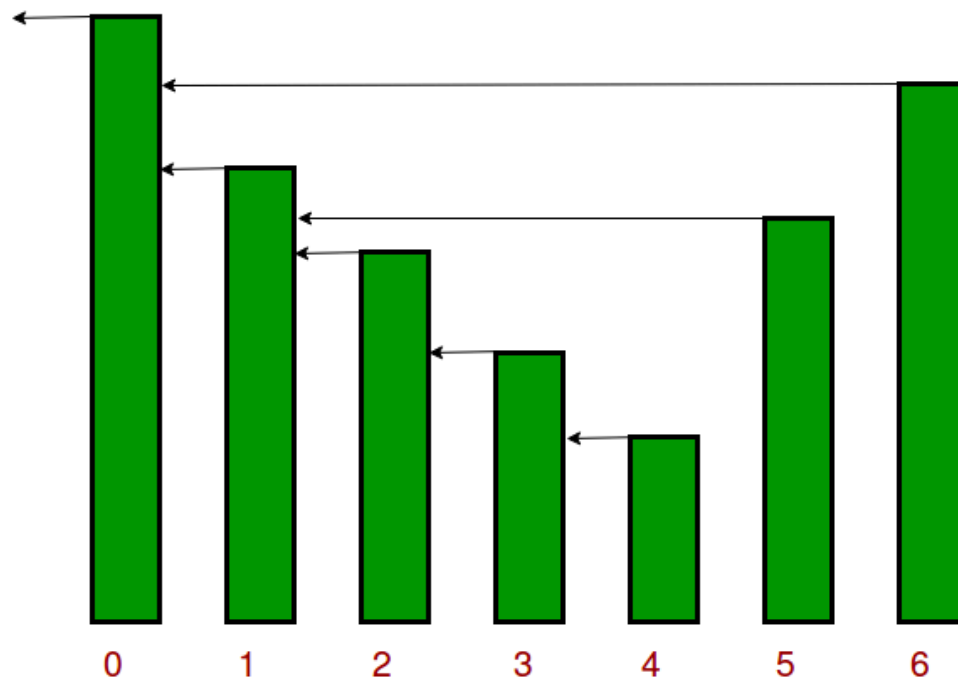
[Expected Approach] Using Stack - $O(n)$ Time and $O(n)$ Space

In the previous approach, for each day we kept moving leftward and comparing prices one by one. We continued as long as the current day's price was greater than or equal to the previous day's price. But the moment we find a price greater than the current day's price, we stop.

That means we are always stopping at the **first previous greater element**.

The idea is to use a stack to directly find this first previous greater element for every day, instead of checking all consecutive smaller ones. Once we know that index, we can compute the span as:

$Span[i] = currentIndex - indexOfPreviousGreaterElement$



C++ Java Python C# JavaScript

```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;

vector<int> calculateSpan(vector<int>& arr) {

    int n = arr.size();
    vector<int> span(n);
    stack<int> st;
```

```

// Process each day's price
for (int i = 0; i < n; i++) {

    // Remove elements from the stack while the current
    // price is greater than or equal to stack's top price
    while (!st.empty() && arr[st.top()] <= arr[i]) {
        st.pop();
    }

    // If stack is empty, all elements to the left are smaller
    // Else, top of the stack is the last greater element's index
    if (st.empty()) {
        span[i] = (i + 1);
    }
    else {
        span[i] = (i - st.top());
    }

    // Push the current index to the stack
    st.push(i);
}

return span;
}

int main() {
    vector<int> arr = {10, 4, 5, 90, 120, 80};
    vector<int> span = calculateSpan(arr);
    for (int x : span) {
        cout << x << " ";
    }
    return 0;
}

```

Output

1 1 2 4 5 1

